

tf.compat.v1.get_variable Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/get_variable

Gets an existing variable with these parameters or create a new one.

Code Examples

```
tf.compat.v1.get_variable(  
    name,  
    shape=None,  
    dtype=None,  
    initializer=None,  
    regularizer=None,  
    trainable=None,  
    collections=None,  
    caching_device=None,  
    partitioner=None,  
    validate_shape=True,  
    use_resource=None,  
    custom_getter=None,  
    constraint=None,  
    synchronization=tf.VariableSynchronization.AUTO,  
    aggregation=tf.compat.v1.VariableAggregation.NONE  
)
```

```
tf.compat.v1.get_variable(  
    name,  
    shape=None,  
    dtype=None,  
    initializer=None,  
    regularizer=None,  
    trainable=None,  
    collections=None,  
    caching_device=None,  
    partitioner=None,  
    validate_shape=True,  
    use_resource=None,  
    custom_getter=None,  
    constraint=None,  
    synchronization=tf.VariableSynchronization.AUTO,  
    aggregation=tf.compat.v1.VariableAggregation.NONE  
)
```

```
tf.VariableSynchronization.AUTO
```

```
tf.compat.v1.VariableAggregation.NONE
```

```
def foo():  
    with tf.variable_scope("foo", reuse=tf.AUTO_REUSE):  
        v = tf.get_variable("v", [1])  
        return v  
v1 = foo() # Creates v.  
v2 = foo() # Gets the same, existing v.  
assert v1 == v2
```

```
def foo():
    with tf.variable_scope("foo", reuse=tf.AUTO_REUSE):
        v = tf.get_variable("v", [1])
    return v
v1 = foo() # Creates v.
v2 = foo() # Gets the same, existing v.
assert v1 == v2
```

```
def custom_getter(getter, name, *args, **kwargs):
    return getter(name + '_suffix', *args, **kwargs)
```

```
def custom_getter(getter, name, *args, **kwargs):
    return getter(name + '_suffix', *args, **kwargs)
```

Documentation

Gets an existing variable with these parameters or create a new one.

Migrate to TF2

Although it is a legacy `compat.v1` api,
`tf.compat.v1.get_variable` is mostly compatible with eager
execution and `tf.function` but only if you combine it with the
`tf.compat.v1.keras.utils.track_tf1_style_variables` decorator. (Though
it will behave as if `reuse` is always set to `AUTO_REUSE`.)

See the
model migration guide
for more info.

If you do not combine it with

`tf.compat.v1.keras.utils.track_tf1_style_variables`, `get_variable` will create a brand new variable every single time it is called and will never reuse variables, regardless of variable names or reuse arguments.

The TF2 equivalent of this symbol would be `tf.Variable`, but note that when using `tf.Variable` you must make sure you track your variables (and regularizer arguments) either manually or via `tf.Module` or `tf.keras.layers.Layer` mechanisms.

A section of the [migration guide](#) provides more details on incrementally migrating these usages to `tf.Variable` as well.

Description

Used in the notebooks

- Migrating model checkpoints
- Use TF1.x models in TF2 workflows
- Linear Mixed-Effect Regression in {TF Probability, R, Stan}
- Graph-based Neural Structured Learning in TFX

This function prefixes the name with the current variable scope and performs reuse checks. See the [Variable Scope How To](#) for an extensive description of how reusing works. Here is a basic example:

If `initializer` is `None` (the default), the default initializer passed in the variable scope will be used. If that one is `None` too, a `glorot_uniform_initializer` will be used. The initializer can also be

a Tensor, in which case the variable is initialized to this value and shape.

Similarly, if the regularizer is None (the default), the default regularizer passed in the variable scope will be used (if that is None too, then by default no regularization is performed).

If a partitioner is provided, a PartitionedVariable is returned.

Accessing this object as a Tensor returns the shards concatenated along the partition axis.

Some useful partitioners are available. See, e.g., `variable_axis_size_partitioner` and `min_max_variable_partitioner`.

Returns

Raises